Lab 2: Sexism Identification in Twitter

Session 1. Machine Learning and Feature Engineering

Goal

The main objective is to train and validate several supervised classification models for addressing the tasks proposed in the shared task *"EXIST2025: sEXism Identification In Social neTworks"*[1]. Firstly, the students will practice supervised machine learning methods such as Support Vector Machines, Logistic Regression, Decision Trees, and Multilayer Perceptrons, applying them to real-world natural language processing problems. This hands-on experience will provide a solid understanding of how these models can tackle complex language challenges. Secondly, the students will explore the benefits and drawbacks of combining multiple machine learning models. They will gain insights into ensemble techniques, including Voting, Stacking, Bagging, Boosting, and Random Forests. By harnessing the collective strength of the simple models, ensembles can achieve a better performance than using isolated models. Finally, the session introduces two methods for exploring the hyperparameter space, a crucial step in achieving optimal model performance. Python library scikit-learn will be used to implement and validate the models.

[1] https://nlp.uned.es/exist2025/

Bibliography

1. Bishop, Christopher M.; Nasrabadi, Nasser M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006. Chapters 4, 5 and 7
2. Duda, Richard O., et al. *Pattern Classification*. John Wiley & Sons, 2006. Chapter 6
3. Fernández, Alberto, et al. *Learning from imbalanced data sets*. Cham: Springer, 2018.
4. Rokach, L. (2019). *Ensemble learning: pattern classification using ensemble methods*. World Scientific Publishing Company Incorporated.
5. *Seni, G., & Elder, J. F. (2010). Ensemble methods in data mining: improving accuracy through combining predictions. Synthesis lectures on data mining and knowledge discovery, 2(1), 1-126.*
6. Theodoridis, Sergios; Koutroumbas, Konstantinos. *Pattern Recognition*. Elsevier, 2006. Chapter 4
7. Zhang, C., & Ma, Y. (Eds.). (2012). *Ensemble machine learning: methods and applications*. Springer Science & Business Media.
8. Zhou, Z. H. (2012). *Ensemble methods: foundations and algorithms*. CRC press.
9. https://scikit-learn.org/stable/modules/svm.html
10. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC

11. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
12. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html#sklearn.tree.DecisionTreeClassifier
13. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifiier
14. https://imbalanced-learn.org/stable/user_guide.html#user-guide
15. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
16. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

EVALUATION MEASURES

Evaluation measures in machine learning are used to quantify the performance of a model in terms of its ability to make accurate predictions on new unseen data. The specific evaluation measures used depend on the type of problem being solved (e.g., classification, regression, clustering) and the desired predictions. The basic idea of evaluating a classifier is to measure the number of cases that are correctly classified and the number of misclassifications. A binary classifier can obtain the following types of predictions:

	Ground Truth (Test data)		
Model (Test data)	**Positive**	**Negative**	**Total**
Positive	TP	FP	TP+FP
Negative	FN	TN	FN+TN
Total	TP+FN	FP+TN	TOTAL

The following terms are used to describe the predictions of a classification algorithm:

- True positives (TP): Examples that belong to the class and are correctly classified by the algorithm.
- True negatives (TN): Examples that do not belong to the class and are correctly classified as not belonging to it.
- False positives (FP): Examples are incorrectly classified as belonging to the class when they do not belong.
- False negatives (FN): Examples incorrectly classified as not belonging to the class when they actually belong to it.

Based on these types of predictions, several evaluation measures can be defined, including **accuracy (Acc)**, **precision (Pr)**, **recall (Rc)**, and **F1-score**.

$$Acc = \frac{TP+TN}{TP+TN+FP+FN}$$

The *Acc* may be inappropriate when dealing with unbalanced data in binary classification. For instance, consider a scenario with 9990

training examples in the positive class and only 10 in the negative class. In such cases, relying solely on *Acc* as a performance metric could be misleading. For instance, if a classifier always predicts the positive class, it will achieve a high accuracy of 99.9%. However, this would not necessarily mean that the classifier effectively captures patterns in the training data or makes accurate predictions for the negative class. Therefore, alternative metrics such as precision, recall, or **F1-score** may be more informative in evaluating the performance of classifiers on unbalanced data.

$$Pr = \frac{TP}{TP+FP} \qquad Rc = \frac{TP}{TP+FN} \quad F_1 = 2\frac{Pr*Rc}{Pr+Rc}$$

Conversely to the *Acc,* the *F1-score* can be calculated by class or overall for the test set. Two ways to compute the overall F1-score are **micro-averaged F1 (*F1-micro*)** or **macro-averaged F1 (*F1-macro*)**.

Micro-averaged F1 is calculated by first computing the overall true positives (TP), false positives (FP), and false negatives (FN) across all classes and then computing the F1 score using these overall values. This approach gives equal importance to each prediction regardless of the class it belongs to. Micro-averaged F1 is useful when you have balanced classes and care more about overall performance than individual classes' performance.

On the other hand, macro-averaged F1 is calculated by computing the F1 score for each class separately and then taking the average across all classes. This approach gives equal importance to each

class, regardless of the number of data points in it. The F1 macro average is especially useful in multiclass, multilabel tasks with an imbalance in the number of samples per class and you want to have a measure of the overall performance of the model giving the same importance to all classes.

In the EXIST2025 shared task, subtask 1 is a binary classification task, while subtask 2 is a multiclass classification class. That is why the F1 score for the positive class (sexism) was considered the official measure in subtask 1. While the macro-averaged F1 was used to evaluate the participating systems in the second subtask. Therefore, we will use these measures to evaluate the effectiveness of the classifiers trained for addressing subtasks 1 and 2.

AUTOMATIC CLASSIFICATION (supervised models)

Supervised classification is a machine learning algorithm that aims to predict a categorical target variable based on a set of input features. The algorithm is trained on a labeled dataset, where the correct output is already known, to learn the relationship between the input features and the target variable. The trained algorithm can then predict the target variable for new input data it has not seen before. The algorithm's accuracy is measured by comparing the predicted output to the ground truth output for the test data. Some popular algorithms for supervised classification include Support Vector Machines, Logistic Regression, Decision Trees, Multilayer Perceptron, etc.

Support Vector Machines (SVMs)

SVMs are a type of machine learning algorithm used for classification and regression tasks. The main idea behind SVMs is to find the best possible boundary or hyperplane to separate two or more classes in a high-dimensional space.

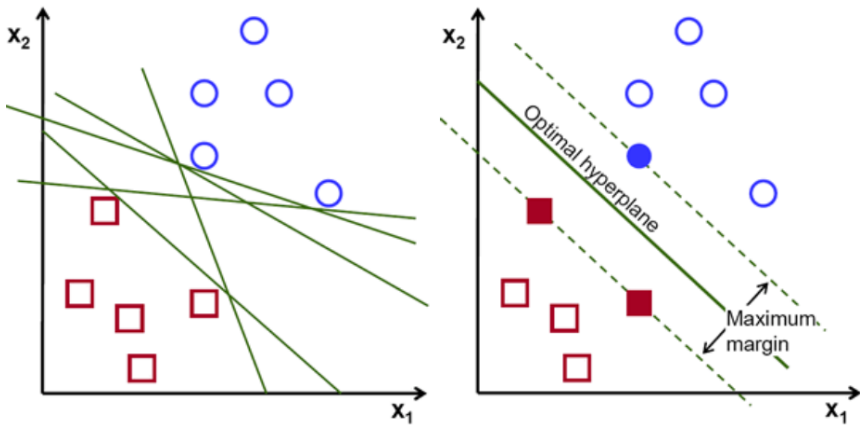

Figure 1. Hyperplane with maximum margin.

To do this, SVMs use a technique called *maximum margin classification*, which aims to find the hyperplane with the largest distance between the data points of the different classes (see Figure 1). The data points closest to the hyperplane on either side are called *support vectors, and* they define the hyperplane.

SVMs work by mapping the input data points into a higher-dimensional space, where finding a hyperplane to separate the data points is more effortless. This is done using kernel functions, which calculate the similarity between data points in the high-dimensional space. Some common kernel functions include Linear, Polynomial, and Radial Basis Functions.

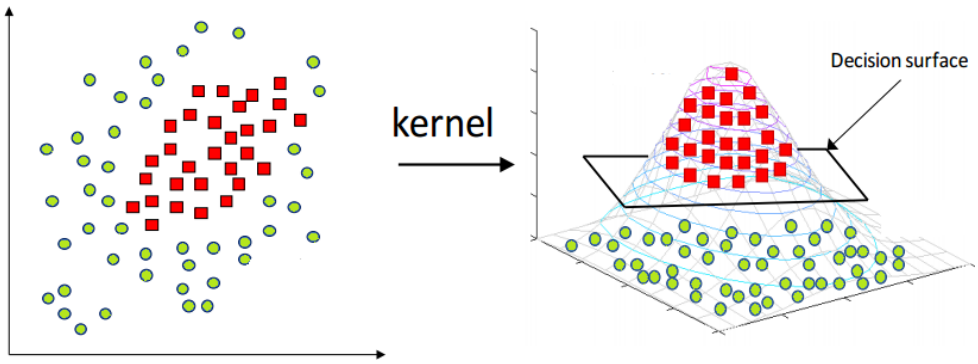

Figure 2. Transformation from a 2D to a 3D space.

Once the hyperplane is found, SVMs can classify new data points based on which side of the hyperplane they fall on. SVMs are helpful for image classification, spam detection, and text classification tasks such as sentiment analysis.

Advantages

- One of the advantages of SVMs is that they are less prone to overfitting than other classification algorithms, such as decision trees or neural networks.
- SVMs offer good accuracy in many problems of classification.
- SVMs use less memory than other classifiers because they use a subset of training points in decision-making.
- SVMs perform well with a clear separation margin and in high-dimensional spaces.

Disadvantages

- SVMs are not suitable for large datasets due to their high training time.
- SVMs are designed to solve binary classification problems.

- SVMs perform poorly with overlapping classes and are also sensitive to the choice of kernel function and the tuning of hyperparameters. Therefore, selecting these parameters to achieve the best performance is essential.

SVC, NuSVC and LinearSVC are methods for binary and multi-class classification on a dataset. SVC and NuSVC are similar but accept slightly different sets of parameters and have different mathematical formulations. On the other hand, LinearSVC is another implementation of Support Vector Classification for the case of a linear kernel. LinearSVC does not accept parameter kernels, which are assumed to be linear.

Hyperparameters

- *Kernel*: Defines the transformation of the input data of the dataset.
- *Regularization*: Reduces overfitting, this hyperparameter reduces the variance of the model parameters. However, it does so at the expense of adding bias to the estimation.
- *Parameter C*: Controls how much errors are tolerable in optimization. A lower value of C creates a hyperplane with a small margin, and a higher value of C makes a hyperplane with a more significant margin.

EXAMPLE 1

```
from sklearn import svm
from sklearn.metrics import f1_score, classification_report

###########################SUBTASK 1##################################
#Using the SVC with default settings
clf1_en = svm.SVC()
clf1_en.fit(X1_train, y1_train)
predicted1 = clf1_en.predict(X1_dev)

print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

###########################SUBTASK 2##################################
clf2_en = svm.SVC()
clf2_en.fit(X2_train, y2_train)
predicted2 = clf2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev, predicted2, digits=4)
print(report)
```

What can we do when we have more than two classes in our classification problems?

Multiclass classification problems are a type of machine learning problem where the goal is to classify data points into one of three or more predefined classes. In contrast to binary classification, where the goal is to classify data points in one of two classes, multiclass classification problems require classifiers to identify multiple outcomes. For this purpose, sklearn implements two

strategies (**One-vs-Rest** and **One-vs-One**) that allow extending the SVC operation to multiple classes.

- **One-vs-Rest (OvR)** approach: In this approach, SVM trains K binary classifiers, where K is the number of classes. Each classifier is trained to distinguish one class from the rest. During testing, each classifier produces a score for each class, and the class with the highest score is selected. This approach can be more efficient than One-v-One, but it may not work well when the classes are imbalanced. In the case of the SVC class, it is sufficient to define the parameter *decision_function_shape*; by default, it has the value *"ovr"* which represents the one-vs-rest approach.
- **One-vs-One (OvO)** approach: In this approach, the SVM trains K(K-1)/2 binary classifiers, where K is the number of classes. Each classifier is trained to distinguish between a pair of classes. Each classifier produces a prediction during testing, and the class with the most votes is selected. This approach works well when the number of classes is small. In case of a tie (between two classes with an equal number of votes), the class with the highest average value is selected. Since it requires adjusting K(K-1)/2 classifiers, this approach is usually slower than OvR, due to its quadratic complexity.

Logistic Regression (LR)

Logistic Regression is a binary classification algorithm that models the probability of a binary outcome (e.g., 0 or 1) as a function of one or more input variables. The model's output is a probability score, which is transformed into a binary prediction by applying a

decision threshold. The algorithm estimates the coefficients of a linear function that maps the input variables to the log-odds of the binary outcome. These coefficients are typically calculated using maximum likelihood estimation.

Advantages

- LR has several advantages, including its simplicity, interpretability, and ability to handle non-linear relationships between the input variables and the outcome.
- LR makes no assumptions about class distributions in feature space.
- LR performs well in many simple datasets, particularly, when the dataset is linearly separable.

Disadvantages

- LR assumes that the relationship between the input variables and the log-odds of the outcome is linear, and it may not work well when the classes are imbalanced or when the input variables are highly correlated.
- If the number of data points is smaller than the number of features, LR should not be used; otherwise, it may lead to overfitting.
- Non-linear problems cannot be solved with logistic regression because it has a linear decision surface.
- Linearly separable data are rarely encountered in real-world scenarios.

Hyperparameters

Penalty: Specify the norm of the penalty by default. The model uses the *L2* norm. Other options are *L1* and *elasticnet*.
Tol: Tolerance for stopping criteria by default=1e-4
Parameter C: Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization.
Solver: Algorithm to use in the optimization problem. Default is *lbfgs*, the other possibilities are *liblinear*, *newton-cg*, *newton-cholesky*, *sag* and *saga*.

EXAMPLE 2

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, classification_report
###########################SUBTASK 1##################################
#Using the LR with default settings
clf1_en = LogisticRegression()
clf1_en.fit(X1_train, y1_train)
predicted1 = clf1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

###########################SUBTASK 2##################################
clf2_en = LogisticRegression()
clf2_en.fit(X2_train, y2_train)
predicted2 = clf2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

Decision Trees (DTs)

Decision tree classifiers are a type of supervised machine learning algorithm used for classification and regression problems. They work by recursively partitioning the input space into regions, each associated with a class label or a predicted value. The algorithm builds a tree-like model where each internal node represents a test on an input variable, and each branch corresponds to the possible outcomes of the test.

Advantages

- DTs have several advantages, including their interpretability and simplicity. They are used as white box models. Trees can be visualized.
- DTs can handle numerical and categorical input variables. Notice that the Sklearn implementation does not support categorical variables.
- DTs capture non-linear relationships between the input variables and the outcome.
- DTs can handle missing and noisy data and are robust to outliers.

Disadvantages

- DTs are prone to overfitting, especially when the tree is deep and complex. Several techniques have been developed to mitigate overfitting, including pruning, ensemble methods (such as random forests and gradient boosting), and regularization.

- DTs can be unstable because slight variations in the data can generate a completely different tree.
- DTs learning algorithms rely on heuristic algorithms to make locally optimal decisions at each node. Such algorithms cannot guarantee a globally optimal decision tree.

Hyperparameters

Criterion: The function to measure the quality of a split. Supported criteria are *gini*, *entropy*, *log_loss* by default, the criterion used is *gini*.
Splitter: The strategy used to choose the split at each node. Supported strategies are *best* to choose the best partition and *random* to choose the best random split.
Max_depth: The maximum depth of the tree. If *None*, nodes are expanded until all leaves are pure or until all leaves contain less than *min_samples_split* samples.
Min_samples_split: The minimum number of samples required to split an internal node; the default value is set to 2.
Min_samples_leaf : The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least *min_samples_leaf* training samples in each left and right branch. This may have the effect of smoothing the model, especially in regression.
Max_features: The number of features to consider when looking for the best split. It can be set to an integer value, float value or *auto*, *sqrt*, and *log2, by default it is set to None.*

EXAMPLE 3

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score, classification_report

###########################SUBTASK 1##################################
#Using the DT with default settings
clf1_en = DecisionTreeClassifier()
clf1_en.fit(X1_train, y1_train)
predicted1 = clf1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

###########################SUBTASK 2##################################
clf2_en = DecisionTreeClassifier()
clf2_en.fit(X2_train, y2_train)
predicted2 = clf2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev, predicted2, digits=4)
print(report)
```

Multilayer Perceptron (MLP)

An MLP is a type of Artificial Neural Network (ANN) that consists of multiple layers of interconnected nodes (or neurons) that process and transform input data into output. The input data is fed into the input layer, which passes the data through one or more hidden layers before producing the outcome in the final output layer. Each neuron in the MLP receives input from the neurons in the previous layer, calculates a weighted sum of the inputs, applies an activation function, and passes the output to the next layer. The weights and

biases of the neurons are learned through a training process that adjusts them to minimize the error between the predicted output and the true output. MLPs are commonly used in classification and regression tasks and are prevalent in applications such as image recognition, speech recognition, and natural language processing.

Advantages

- MLPs can learn non-linear relationships between inputs and outputs, making them useful for complex problems where linear models may not be sufficient.
- MLPs can be used for both classification and regression tasks and can handle a variety of input.
- MLPs can handle substantial amounts of data and can be scaled up by adding more layers or neurons, allowing them to handle more complex tasks.
- MLPs can generalize from training data to unseen data, making them suitable for tasks like pattern recognition and prediction.
- MLPs can adapt to changing inputs and can learn from new data without requiring retraining from scratch.
- MLPs can be trained and run on parallel computing systems, allowing for faster processing of substantial amounts of data.

Disadvantages

- MLPs can take a long time to train, especially for large and complex datasets. The training process can be computationally expensive and may require a lot of resources.

- MLPs can be prone to overfitting, where the model learns to fit the training data too closely and does not generalize well to new data. Regularization techniques can be used to address this issue.
- MLPs can get stuck in local optima during the training process, where the model converges to a suboptimal solution instead of the global optimum. This can be addressed by using various optimization algorithms and techniques.
- MLPs can be difficult to interpret and understand, making it challenging to explain how the model makes predictions.

Hyperparameters

Hidden_layer_sizes : array-like of shape(*n_layers - 2*,), *default=(100,).* The *ith*-element represents the number of neurons in the *ith*-hidden layer.
Activation: Activation function for the hidden layer, *default=relu*. Other activation functions are *identity*, *logistic*, *tanh* and *relu*.
Solver: The solver for weight optimization, default=*adam*. Other solvers are *lbfgs* and *sgd.*
Batch_size: Size of minibatches for stochastic optimizers, *default=auto*. If the solver is *lbfgs*, the classifier will not use minibatch. When set to *auto*, *batch_size=min(200, n_samples)*.
Learning_rate: Learning rate schedule for weight updates, *default=constant*. Other schedules are *invscaling* and *adaptive.*
Learning_rate_init: The initial learning rate used, *default=0.001*. It controls the step-size in updating the weights. Only used when solver=*sgd* or *adam*.

Max_iter: Maximum number of iterations, *default=200*. The solver iterates until convergence. For stochastic solvers (*sgd* and *adam*), note that this determines the number of epochs (how many times each data point will be used), not the number of gradient steps.
Tol: Tolerance for the optimization, *default=1e-4*. When the loss or score is not improving by at least *tol* for *n_iter_no_change* consecutive iterations, unless *learning_rate* is set to adaptive, convergence is reached and training stops.
Early_stopping: Whether to use early stopping to terminate training when validation score is not improving, default=False. If set to True, it will automatically set aside 10% of training data as validation and terminate training when validation score is not improving by at least *tol* for *n_iter_no_change* consecutive epochs. The split is stratified, except in a multilabel setting. If early stopping is False, then the training stops when the training loss does not improve by more than tol for *n_iter_no_change* consecutive passes over the training set. It is only effective when solver=*sgd* or *adam*.
Validation_fraction: The proportion of training data to set aside as validation set for early stopping, *default=0.1*. Must be between 0 and 1. Only used if *early_stopping* is True.
N_iter_no_change: Maximum number of epochs to not meet *tol* improvement, *default=10*. Only effective when solver=*sgd* or *adam*.

EXAMPLE 4

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import f1_score, classification_report

##########################SUBTASK 1##################################
#Using the MLP with default settings
clf1_en = MLPClassifier(random_state=1, max_iter=300)
clf1_en.fit(X1_train, y1_train)
predicted1 = clf1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
```

```
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

###########################SUBTASK 2#################################
clf2_en = MLPClassifier(random_state=1, max_iter=300)
clf2_en.fit(X2_train, y2_train)
predicted2 = clf2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

ENSEMBLE OF MODELS

Ensemble methods can often outperform simple classifiers due to the concept of the *"wisdom of the crowd"* or *"collective intelligence".* By combining the predictions of multiple models, ensemble methods can mitigate the individual weaknesses of each model and produce more accurate and robust predictions. One theoretical foundation for why ensemble methods work well is the *"bias-variance tradeoff"*. In machine learning, the goal is to build a model that can accurately generalize to new data points that have not been seen before. The *"bias-variance tradeoff"* refers to the tradeoff between a model's ability to fit the training data (low bias) and generalize on new data (low variance).

Simple models such as decision trees tend to have high bias and low variance, meaning they are likely to underfit the training data

but have low variance in their predictions. On the other hand, complex models such as neural networks tend to have low bias but high variance, meaning that they are likely to overfit the training data and have high variance in their predictions. Ensemble methods can help to balance the "bias-variance tradeoff" by combining multiple models with different biases and variances. For example, bagging and random forest methods use multiple models to create an ensemble with lower variance and better generalization performance. Boosting techniques such as AdaBoost and Gradient Boosting can improve bias and variance by iteratively adding new models that focus on correcting the errors of previous models.

Approaches to combine models

The most common approaches for supervised ensemble models in machine learning are:

1. **Voting**: In this approach, multiple base models are trained on the same dataset, and the final prediction is made by taking a majority vote of the predictions of the single models. This approach is typically used when the base models are diverse and complementary.
2. **Bagging**: In this approach, multiple instances of the same base model are trained on different subsets of the training data. The final prediction is made by averaging the predictions of the individual models. Bagging is typically used when the base model has high variance or is unstable.

3. **Boosting**: In this approach, multiple base models are trained sequentially, with each subsequent model learning from the errors of the previous model. The final prediction is made by taking a weighted average of the predictions of the single models. Boosting is typically used when the base model is weak, aiming to improve accuracy and reduce bias.
4. **Stacking**: In this approach, multiple base models are trained on the same dataset, and the predictions of the individual models are used as input features for a final model. The final model learns to combine the predictions of the single models to improve accuracy. Stacking is typically used when the base models are diverse and complementary.

VOTING

The idea is that by combining the predictions of multiple models that are trained on the same data but using different algorithms or parameter settings, the resulting prediction will be more accurate and robust than any single model's prediction.

There are two main types of voting ensembles: *"hard voting"* and *"soft voting."* In hard voting, the ensemble's prediction is determined by a simple majority vote of the base models' predictions. For example, if three base models predict that an input belongs to the class *sexism*, and two predict that it belongs to the class *non-sexism*, then the ensemble would predict the class *sexism*. In soft voting, the ensemble prediction is based on the average probabilities of the base models' predictions for each class.

The class with the highest average probability is then selected as the final prediction.

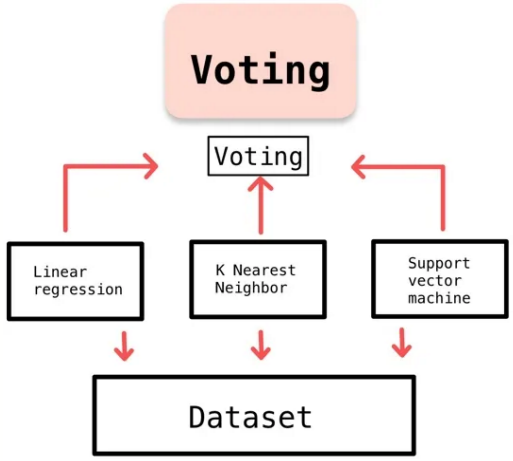

Figure 3. Voting ensemble: hard voting

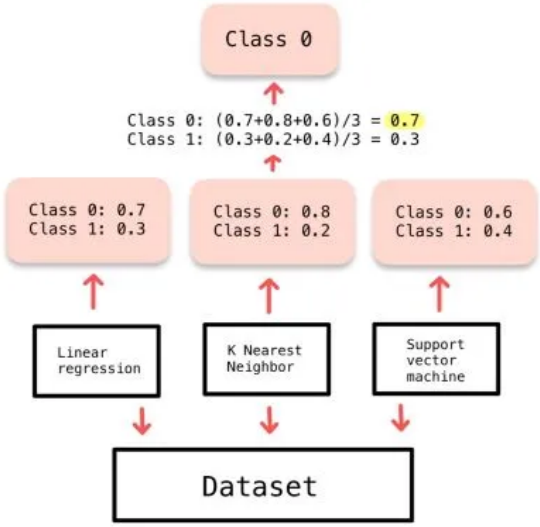

Figure 4. Voting ensemble: soft voting

Voting ensembles can be applied to a wide range of machine learning tasks, including classification, regression, etc. They can be particularly useful in situations where the performance of a single model is limited or where different models have different strengths and weaknesses. By combining the predictions of multiple models, voting ensembles can provide a more accurate

and robust prediction that is less likely to be affected by individual model biases or errors.

EXAMPLE 5

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import f1_score, classification_report

###########################SUBTASK 1##################################
# create the base models, Logistic Regression, Decision Tree, and Support Vector Machine
model1 = LogisticRegression()
model2 = DecisionTreeClassifier()
model3 = SVC()
# create the voting ensemble
ensemble1_en = VotingClassifier(estimators=[('lr', model1), ('dt', model2), ('svm', model3)], voting='hard')
# train the ensemble
ensemble1_en.fit(X1_train, y1_train)
predicted1 = ensemble1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

###########################SUBTASK 2##################################
# create the voting ensemble
ensemble2_en = VotingClassifier(estimators=[('lr', model1), ('dt', model2), ('svm', model3)], voting='hard')
ensemble2_en.fit(X2_train, y2_train)
predicted2 = ensemble2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

The voting parameter is set to *'hard'* to indicate that we want to use hard voting.

BAGGING

Bagging is based on bootstrap resampling, which involves randomly sampling the training data with replacement to create multiple subsets of the same size as the original data set. The basic idea behind bagging is to train a set of base models on different bootstrap samples of the training data and then aggregate their predictions using a simple averaging or voting scheme to create the final prediction. This can help reduce the model's variance and improve its generalization performance.

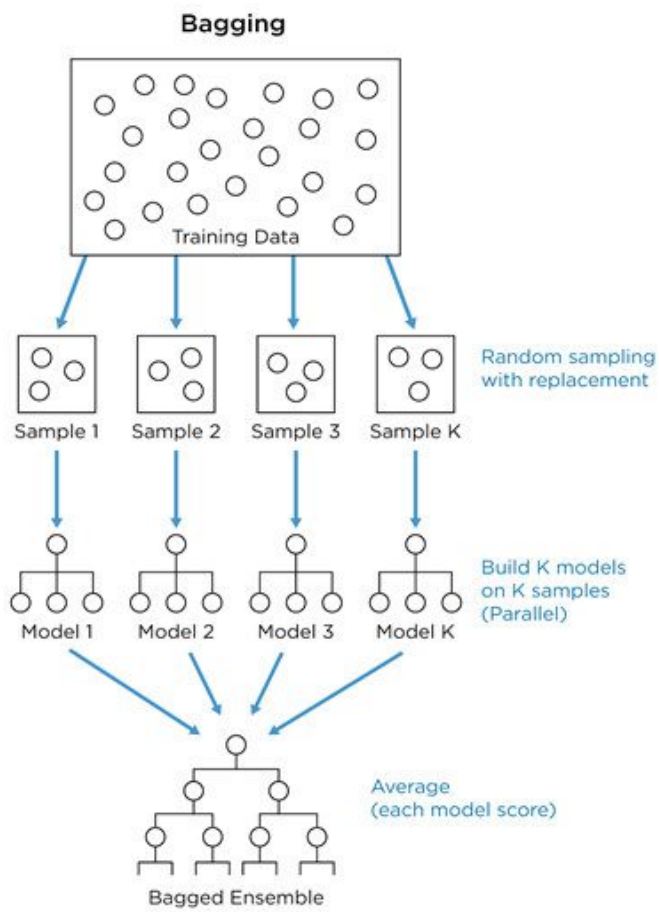

Figure 5. General schema of the bagging ensemble

The bagging algorithm can be summarized as follows:

1. Create multiple bootstrap samples of the training data, each of the same size as the original data set.
2. Train a base model on each bootstrap sample.
3. Aggregate the predictions of the base models using a simple averaging or voting scheme to create the final prediction.

Random Forest is an extension of bagging. In this approach, multiple decision trees are trained on different subsets of the training data. It also takes the random selection of features rather than using all features to grow trees. The final prediction is made by averaging the predictions of the individual trees. Random forests are typically used when the dataset has high dimensionality and complex relations among the features.

EXAMPLE 6

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score, classification_report

###########################SUBTASK 1##################################
# create the voting ensemble
ensemble1_en = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=10)
# train the ensemble
ensemble1_en.fit(X1_train, y1_train)
predicted1 = ensemble1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)
```

```
############################SUBTASK 2##################################
# create the voting ensemble
ensemble2_en = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=10)
ensemble2_en.fit(X2_train, y2_train)
predicted2 = ensemble2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

EXAMPLE 7

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, classification_report

############################SUBTASK 1##################################
# create the voting ensemble
ensemble1_en = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=0)
# train the ensemble
ensemble1_en.fit(X1_train, y1_train)
predicted1 = ensemble1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

############################SUBTASK 2##################################
# create the voting ensemble
ensemble2_en = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=0)
ensemble2_en.fit(X2_train, y2_train)
predicted2 = ensemble2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

BOOSTING

Boosting is an ensemble learning method combining multiple weak learners to create a strong learner. Conversely to bagging, which trains each weak learner independently, boosting trains weak learners sequentially, with each subsequent learner concentrating on the data points the previous learners misclassified. For that, the boosting method reweights the training data iteratively so that the next model focuses more on the data points that the previous model misclassified. AdaBoost and XGBoost are two boosting approaches.

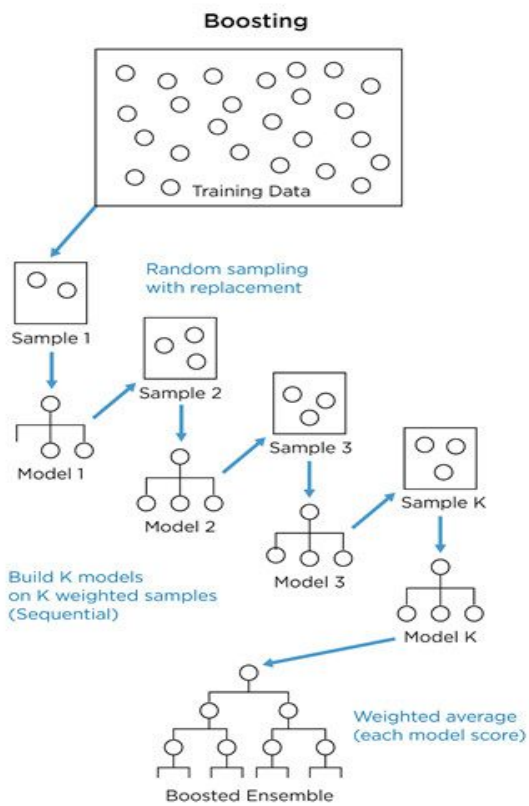

Figure 6. General schema of the boosting ensemble

AdaBoost (Adaptive Boosting) is a popular boosting algorithm introduced by Freund and Schapire in 1997. The AdaBoost algorithm works by iteratively adding weak learners to strong learners. At each iteration, the algorithm selects a weak learner that can best classify the examples that the previous weak learners misclassified. The final model is a weighted combination of the weak learners, where the weights are determined by the accuracy of each weak learner on the training data.

The mathematical foundation of AdaBoost can be described as follows. Suppose we have a dataset with input features *X* and corresponding target values *y*, where y is a binary classification label (+1 or -1). We also have a set of base models, denoted by *M={Model1, Model2, ..., Modelk}*, which can be used to make binary predictions on X. AdaBoost aims to train a weighted combination of the base models that can make accurate binary predictions on *X*.

EXAMPLE 8

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, classification_report

###########################SUBTASK 1##################################
# Create the boosting classifier
ensemble1_en=AdaBoostClassifier(estimator=LogisticRegression(),n_estimators=10)
# train the ensemble
ensemble1_en.fit(X1_train, y1_train)
predicted1 = ensemble1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)
```

```
###########################SUBTASK 2##################################
# Create the boosting classifier
ensemble2_en=AdaBoostClassifier(estimator=LogisticRegression(),n_estimators=10)
ensemble2_en.fit(X2_train, y2_train)
predicted2 = ensemble2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

XGBoost (eXtreme Gradient Boosting) is a robust ensemble learning algorithm introduced by Chen and Guestrin in 2016. It extends the gradient boosting method, focusing on scalability and speed. XGBoost is widely used in machine learning competitions and is known for its high accuracy and performance. The XGBoost method minimizes a differentiable loss function, such as mean squared error or log loss, using gradient descent. XGBoost iteratively adds decision trees to a weighted combination of previous trees, where the weights are determined by the negative gradient of the loss function.

EXAMPLE 9

```
!pip install xgboost
import xgboost as xgb
import numpy as np
from sklearn.metrics import f1_score, classification_report

###########################SUBTASK 1##################################
# Convert data to DMatrix format
dtrain1_en = xgb.DMatrix(X1_train, label=y1_train)
dtest1_en = xgb.DMatrix(X1_dev, label=y1_dev)

# Set XGBoost parameters
params = {
'max_depth': 3,
'eta': 0.1,
```

```
'objective': 'binary:logistic',
'eval_metric': 'auc'
}

# Train the model
num_rounds = 100
bst_en = xgb.train(params, dtrain1_en, num_rounds)
# Make predictions on the test set
predicted1 = bst_en.predict(dtest1_en)
predicted1 = np.asarray([int(round(value)) for value in predicted1])
# Evaluate the model
print("Predicted\t-->\t", predicted1)
print("GroundTruth\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)


###########################SUBTASK 2##################################
# Convert data to DMatrix format
dtrain2_en = xgb.DMatrix(X2_train, label=y2_train)
dtest2_en = xgb.DMatrix(X2_dev, label=y2_dev)

# Set XGBoost parameters
params = {
'max_depth': 3,
'eta': 0.1,
"num_class": 3,
'objective': 'multi:softmax',
'eval_metric': 'auc'
}

# Train the model
num_rounds = 100
bst2_en = xgb.train(params, dtrain2_en, num_rounds)
# Make predictions on the test set
predicted2 = bst2_en.predict(dtest2_en)
predicted2 = np.asarray([int(round(value)) for value in predicted2])
# Evaluate the model
print("Predicted\t-->\t", predicted2)
print("GroundTruth\t-->\t", y2_dev)
predicted2 = np.asarray([int(round(value)) for value in predicted2])
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

STACKING

Stacking, or stacked generalization, is an ensemble learning technique that combines multiple base models to improve prediction performance. The basic idea behind stacking is to train a meta-model on the predictions of several base models, hoping that the meta-model can learn to combine the strengths of the base models and mitigate their weaknesses.

The mathematical foundations of stacking can be described as follows. Suppose we have a dataset with input features X and corresponding label values y. We have a set of base models, denoted by $M=\{Model1, Model2, ..., Modelk\}$, which can be used to make predictions on X. Stacking aims to train a **meta-model**, denoted by M', that can make more accurate predictions on X than each individual base model in M.

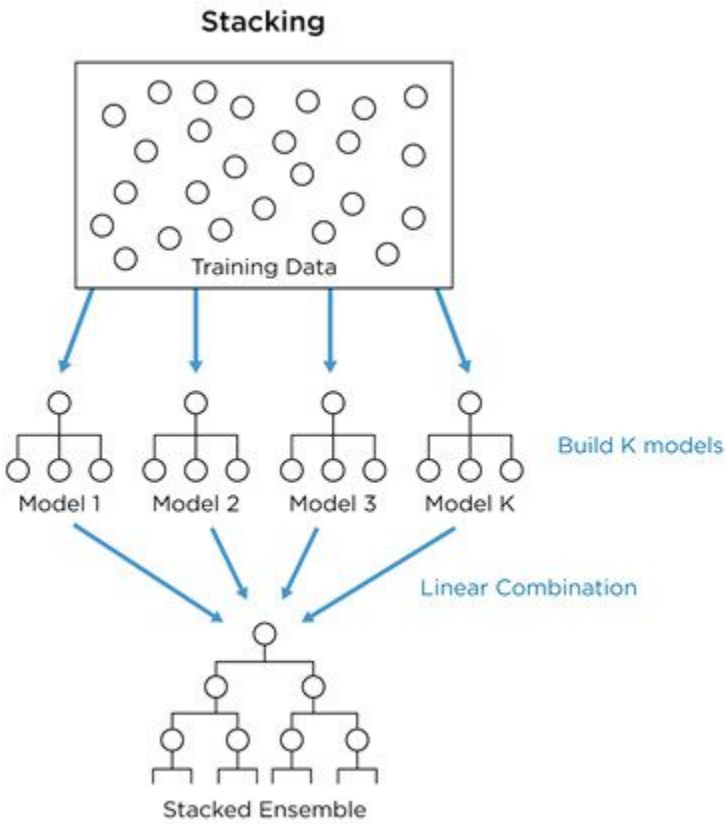

Figure 7. General schema of the stacking ensemble

Firstly, we first split the data into training and testing sets to do this. We then use the training set to train the base models in *M* and use them to make predictions on the testing set. We stack these predictions together to form a new feature matrix, denoted by *Z*, where each row corresponds to a sample in the testing set, and each column corresponds to the predicted value of the test set by one of the base models in *M*. Then, we use *Z* as the new input features to train *M'*. The target values for *M'* are the true label values of the testing set. *M'* can be any model type, such as linear regression, logistic regression, or neural network. Once *M'* has been trained, we can predict new data points.

Stacking is a robust technique for improving prediction performance in machine learning, but it should be used wisely and carefully considering the computational costs. Moreover, the meta-learner in stacking techniques can create a "black box" aggregation mechanism that complicates comprehensive model interpretation.

EXAMPLE 10

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import f1_score, classification_report
###########################SUBTASK 1##################################
# create the base models,K-NNeighbors, Logistic Regression, Decision Tree, and Support Vector Machine
base_models_en = [('dt', DecisionTreeClassifier()), ('svm', SVC()), ('knn', KNeighborsClassifier())]
# Create the meta-model
meta_model_en = LogisticRegression()
# Create the boosting classifier
ensemble1_en=StackingClassifier(estimators=base_models_en, final_estimator=meta_model_en)
# train the ensemble
ensemble1_en.fit(X1_train, y1_train)
predicted1 = ensemble1_en.predict(X1_dev)
print("Predicted task1\t-->\t", predicted1)
print("GroundTruth task1\t-->\t", y1_dev)
#Computing the F1 Score for the positive class (sexism), subtask 1
f1_positive = f1_score(y1_dev, predicted1, pos_label=1)
print(f"F1-score (Positive Class): {f1_positive}")
#Obtainig a detailed classification report
report = classification_report(y1_dev,predicted1, digits=4)
print(report)

###########################SUBTASK 2##################################
# Create the boosting classifier
ensemble2_en=StackingClassifier(estimators=base_models_en, final_estimator=meta_model_en)
ensemble2_en.fit(X2_train, y2_train)
predicted2 = ensemble2_en.predict(X2_dev)
print("Predicted task2\t-->\t", predicted2)
print("GroundTruth task2\t-->\t", y2_dev)
#Computing the macro-average F1 for the second subtask
f1_macro = f1_score(y2_dev, predicted2, average='macro')
print(f"F1-score (Macro-Averaged): {f1_macro}")
#Obtainig a detailed classification report
report = classification_report(y2_dev,predicted2, digits=4)
print(report)
```

HYPERPARAMETERS SETTING

Hyperparameters are not learned directly in the classifier. It is possible and recommended to search the hyperparameter space for better results. Two generic approaches can be used for parameter search:

Grid Search exhaustively considers all parameter combinations.

EXAMPLE 11

```
from sklearn import svm
from sklearn.metrics import f1_score, make_scorer
f1_scorer = make_scorer(f1_score, pos_label=1)
from scipy.stats import uniform, randint
from sklearn.model_selection import GridSearchCV

param_grid = {
'C': [0.1, 2],
'gamma': [0,1], # Uniform distribution for gamma
'kernel': [ 'rbf', 'poly'],
'class_weight': ['balanced'],
'degree': [5]
}

clf1 = GridSearchCV(estimator=svm.SVC(), param_grid=param_grid, scoring=f1_scorer, n_jobs=-1, verbose=1)
clf1.fit(X1_train, y1_train)
#Obtaining the best hyper-parameters
print(clf1.best_params_)
#Obtain the best classifier
print(clf1.best_estimator_)
```

Randomized Search: samples a parameter space with a specific distribution.

EXAMPLE 12

```
from scipy.stats import uniform
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import matthews_corrcoef, make_scorer
f1macro_scorer = make_scorer(f1_score, average='macro')

mysvm= svm.SVC()
param_distributions = {
'C': uniform(0.1, 10), # Uniform distribution between 0.1 and 10
'gamma': uniform(0.0001, 1), # Uniform distribution for gamma
'kernel': ['linear', 'rbf', 'poly'],
'class_weight': [None, 'balanced'],
'degree': randint(1, 5) # Random integer between 1 and 5
}
clf = RandomizedSearchCV(mysvm, param_distributions, scoring=f1macro_scorer, n_jobs=-1, verbose=1)
clf.fit(X1_train, y1_train)
print(clf.best_params_)
print(clf.best_estimator_
```

GridSearchCV and RandomSearchCV have successive equivalents for halving the search: HalvingGridSearchCV and HalvingRandomSearchCV.

ACTIVITY

Students must train and validate machine learning models for the EXIST2025 shared task. Students must address subtasks 1 and 2.

1. To distinguish sexist tweets from non-sexist ones (binary classification problem).
2. To categorize the sexist tweets according to the intention of the author, the intention labels are: DIRECT, REPORTED, and JUDGEMENTAL (a multiclass classification problem)

For that purpose and for each subtask:

1. Students must train and evaluate **at least <u>two classifiers</u>** and compare their results. One of the classifiers can be an ensemble of simple models.
2. Students must select **at least <u>two text representation methods</u>** studied in previous practices (traditional forms, static embedding-based, and contextual embedding-based) and train the selected models using those representations. Preprocess the text as deemed appropriate for each representation method.
3. Students must train and evaluate models for both subtasks (1 and 2) on the English and Spanish datasets.

Altogether, at least **two preprocessing methods, two text representation methods, and two classification models** must be selected, used, and evaluated for each subtask and language. <u>Students are encouraged to test more combinations</u> and try to get the best possible results.

The trained models' performance must be evaluated based on the official evaluation measures. In the case of subtask 1, the F1 of the positive class (sexism) must be reported. In the case of the second subtask, the F1 macro-average must be reported. The official training, dev and test set for both languages are in the EXIST-2025-Tweet-Dataset.zip

The **notebook with the source code** and a **short report** explaining the preprocessing applied, the text representations, the classification models, the hyperparameters considered, and the results obtained should be delivered before the next session.

APPENDIX

To read the EXIST-2025-Tweet-Dataset and obtain the hard labels for each task, the *EXISTReader* class can be utilized. This class receives the path to an EXIST-2025-Tweet-Dataset file as a parameter. Accessing different language and subtask labels is accomplished through the *get* method. To retrieve a specific subset of the file, students must specify two key parameters: the language (*lang*) and the subtask (sub*task*). For example, calling the method with lang="EN" and subtask="task2" will return a tuple containing three elements: the dataset IDs, the corresponding texts, and the hard labels as defined by the EXIST2025 shared task organizers.

EXAMPLE 13

```
#Reading the entire dataset for both languages and considering only the hard labels. In this lab we do not address the sexism identification task from a Learning with Disagreement (LwD) perspective.
from readerEXIST2025 import EXISTReader

reader_train = EXISTReader("EXIST_2025_Dataset_V0.2/EXIST2025_training.json")
reader_dev = EXISTReader("EXIST_2025_Dataset_V0.2/EXIST2025_dev.json")

EnTrainTask1, EnDevTask1 = reader_train.get(lang="EN", subtask="1"), reader_dev.get(lang="EN", subtask="1")
EnTrainTask2, EnDevTask2 = reader_train.get(lang="EN", subtask="2"), reader_dev.get(lang="EN", subtask="2")

SpTrainTask1, SpDevTask1 = reader_train.get(lang="ES", subtask="1"), reader_dev.get(lang="ES", subtask="1")
SpTrainTask2, SpDevTask2 = reader_train.get(lang="ES", subtask="2"), reader_dev.get(lang="ES", subtask="2")
```